HANDS-ON LAB MANUAL · VOLUME 3

Installing the Wazuh Agent on Windows Server and Windows Client

Turning two unmonitored Windows machines into a red/blue lab you can actually test against — and understanding every step.

What you will do	Install the Wazuh agent on a Windows Server 2022 VM and a Windows 11 client VM, enrol both into your existing Wazuh manager, then confirm you can see them in the dashboard
Why it matters	Right now your attacker VM is monitored and your two Windows targets are blind. You cannot detect what you cannot see, and you cannot close a gap you never measured.
Lab network	VirtualBox `intnet`, `10.0.0.0/24`, fully isolated from the host
Manager	SIEM-Ubuntu, Wazuh 4.14.8, `10.0.0.40`
Targets	WinServer 2022 `10.0.0.50` · WindowsClient (Win11)
Attacker	Kali `10.0.0.30` (already enrolled as agent 001)
Time	About 15 minutes for the first machine, 5 for the second
Scope	Your own isolated lab VMs only

> **Read this first.** Everything in this document runs against VirtualBox machines on your own laptop, on a network segment that is not reachable from your physical network or the internet. That is what makes it safe to practise offensive technique here. Keep it that way: if you ever point these steps at a machine you do not own, that is a different activity with different rules.

0 Quick start — the whole thing on one page

If you just want it working, this page is enough. Everything after it explains why each step is the way it is.

0.1 Files you need

File	Purpose
`Install-WazuhAgent.ps1`	The installer you run on each Windows VM. Self-contained, no other files needed.
`red-blue-agent-install.html`	Source of this document, if you want to edit it.
`RedBlue-Lab-Vol3-Wazuh-Agent-Windows-Install.pdf`	This document.
`style.css`	Shared print styling used to build the PDF.
`build_pdfs.py`	Renders the HTML to PDF using headless Edge.
`check_html.py`	Validates the HTML: tag balance, entities, table widths.
`verify_pdfs.py / validate.py`	Checks the built PDF for page count, text and content.
`probe_msi.py`	Dumps the real MSI property names out of the installer.

All of the above live in `C:\Users\pc\Desktop\RedBlue-Lab`.

0.2 The five steps

#	Do this	Where
1	Power on the VMs	Host: `VBoxManage startvm "SIEM-Ubuntu" --type headless` and the same for Kali. Confirm Wazuh is up on SIEM-Ubuntu.
2	Give the Windows VMs internet	Host, VMs powered off: `VBoxManage modifyvm "WinServer" --nic2 nat --nictype2 82540EM` and the same for WindowsClient.
3	Run the script on WinServer	Admin PowerShell on the VM, see 0.3.
4	Run the script on WindowsClient	Admin PowerShell on the VM, see 0.3.
5	Confirm in the dashboard	Agents → both machines Active within about 30 seconds. See section 4.

0.3 The two commands you actually type

```
# on WinServer 2022 (10.0.0.50)
powershell -ExecutionPolicy Bypass -File .\Install-WazuhAgent.ps1 `
  -AgentName winsrv-01 -AgentGroup windows,servers

# on WindowsClient (Win11)
powershell -ExecutionPolicy Bypass -File .\Install-WazuhAgent.ps1 `
  -AgentName wincli-01 -AgentGroup windows,endpoints
```

If the script is not in the current directory, give the full path:

```
powershell -ExecutionPolicy Bypass `
  -File C:\Users\pc\Desktop\RedBlue-Lab\Install-WazuhAgent.ps1 `
  -AgentName winsrv-01 -AgentGroup windows,servers
```

If PowerShell refuses to run it, the file may be blocked. Unblock it once:

```
Unblock-File .\Install-WazuhAgent.ps1
```

0.4 How to undo everything

Change	How to revert
The NAT adapter	`VBoxManage modifyvm "WinServer" --nic2 none` (and WindowsClient). The agent keeps working; it only needed the internet to download itself.
The agent on a VM	Apps → Installed apps → Wazuh Agent → Uninstall, or:
	`msiexec /x {840F372A-2C20-49B9-A164-D8A14E8E8D47} /quiet`
Leftover files	`Remove-Item "C:\Program Files (x86)\ossec-agent" -Recurse -Force`
The firewall rule	`Get-NetFirewallRule -DisplayName 'Wazuh Agent 1514' | Remove-NetFirewallRule`
The agent in the dashboard	Agents → row → … → Delete. This removes its history too.

> **Get the product code before you uninstall, do not assume it**
>
> The GUID above is the one from the 4.14.8 MSI, which is what the script downloads. Confirm yours matches before relying on it, because it changes between versions:
>
> ```
> Get-ItemProperty HKLM:\SOFTWARE\WOW6432Node\Microsoft\Windows\CurrentVersion\Uninstall* |
>   Where-Object DisplayName -eq 'Wazuh Agent' | Select-Object DisplayVersion, PSChildName
> ```

0.5 If it goes wrong, look here first

Symptom	Immediate cause
`Service 'Wazuh' does not exist`	The service is **WazuhSvc**, not Wazuh. Every guide gets this wrong.
`MSI returns 0 but nothing installed`	Argument list built into a variable named `$args`. See 5.3.
`ERROR (1208) enrolment`	Port 55000 unreachable. Start the manager, or check the manager IP.
`Download hangs`	No internet route. Add the NAT adapter, or use `-SkipDownload`.
`Installed but not in the dashboard`	Wrong manager IP, or the agent is using the NAT address instead of 10.0.0.x.

1 The gap you are closing

Before installing anything, it helps to be precise about what is missing, because "install an agent" is a vague goal and vague goals produce half-finished labs.

Machine	Role	Status today	After this lab
SIEM-Ubuntu	Wazuh manager	Running, agent 001	Unchanged. It is the brain; it already works.
Kali	Attacker	Enrolled, active	Unchanged. Your red side already reports.
WinServer	Server target	No agent	Enrolled. Every login, process and file change is visible.
WindowsClient	Client target	No agent	Enrolled. Same coverage, endpoint-class rules.

> **The asymmetry that makes this lab worth building**
>
> Your attacker machine is monitored but your targets are not. That is the exact inverse of a real breach, where the victim is instrumented and the attacker's own box usually is not. As it stands, if you run an attack from Kali against WinServer, the only machine that will record it is Kali. The server you are compromising generates no telemetry at all.
>
> Enrolling the two Windows machines turns that around. Now the same attack produces evidence on both ends, and you get to practise the skill that actually matters: reading the victim's logs and working out what happened.

1.1 What the agent actually is, in plain terms

A Wazuh agent is a small Windows service, about 15 MB installed, that does three things continuously:

1. **Reads** security-relevant things on the host: Windows event logs, running processes, listening ports, installed software, file integrity changes, and security configuration.
2. **Evaluates** each event against thousands of built-in detection rules, locally.
3. **Ships** only what matters, encrypted, to your manager. A normal morning generates a trickle of data. A port scan or a failed password spray generates alerts.

Because the filtering happens on the endpoint, you are not shipping raw event logs across the network. That is the single most important architectural idea to understand: **the agent is a filter, not a pipe.**

1.2 Ports involved

Port	Purpose	Direction and who initiates
`1514`	Events and alerts	Agent **connects out** to the manager. Outbound is allowed by default, so no inbound rule is strictly required.
`55000`	Enrolment (authd)	Agent connects out to request a key at first start. Must be reachable or the agent installs but never registers.
`443`	Wazuh dashboard	Your browser, on the manager. This is what you will actually look at.

> **Do not open 55000 or 9200 to your lab subnet**
>
> 55000 is the enrolment service and 9200 is the indexer API. Neither needs to be reachable from the machines you monitor. Opening them broadly is a real and extremely common mistake, and it shows up as a finding in most security assessments. Your guide material makes the same point; it is worth internalising rather than copying.

2 One change to your VMs, and why

There is a practical problem to solve before the install can work, and understanding it teaches something about how VirtualBox networking actually works.

2.1 The problem

Your four VMs sit on a VirtualBox **internal network** (`intnet`), which is `10.0.0.0/24` . This is a deliberate and good choice: a VirtualBox internal network can talk to other VMs on the same internal network and to nothing else. No route to your home LAN, no route to the internet, no route to any real machine.

WinServer and WindowsClient originally had a single network adapter on that internal network and nothing else. Which means:

- They can talk to the manager at `10.0.0.40` . Good, that is the path events travel.
- They **cannot reach the internet**, so they cannot download the Wazuh installer from `packages.wazuh.com` .

> **This bites almost everyone, and the symptom is confusing**
> The agent installer needs to come from somewhere. Without an internet route the download silently hangs or fails, and people conclude the Wazuh site is down. It is not. The VM simply has no route to the internet, by design.

2.2 The change: a second adapter on NAT

The fix is one VirtualBox setting. Each Windows VM keeps its internal-network adapter exactly as it was, and gains a **second** adapter on NAT:

```
# what the VMs look like now
WinServer      nic1 = intnet   10.0.0.50   lab traffic, isolated
               nic2 = nat      10.0.2.15   internet, via the host

WindowsClient  nic1 = intnet   10.0.0.x    lab traffic, isolated
               nic2 = nat      10.0.2.15   internet, via the host
```

Run this on the host, with each VM powered off:

```
VBoxManage modifyvm "WinServer"     --nic2 nat --nictype2 82540EM
VBoxManage modifyvm "WindowsClient" --nic2 nat --nictype2 82540EM
```

To undo it later and return the lab to full isolation:

```
VBoxManage modifyvm "WinServer"     --nic2 none
VBoxManage modifyvm "WindowsClient" --nic2 none
```

> **Why this does not weaken your lab**
>
> The lab segment is still fully isolated. NAT gives the VM a route out to the internet through the host's connection; it does **not** give anyone a route *into* `10.0.0.0/24`. Your Kali box still cannot reach anything except the other lab machines. Attack tooling cannot reach your real network through this.
>
> The practical effect is narrow and specific: these two VMs can now download software, and the agent reports to the manager over the same isolated path it always would have. If you want zero internet route at all once the agent is installed, set the second adapter back to `none` and the agent keeps working, because it only ever needed the internet to fetch itself.

2.3 How to confirm the change took

```
VBoxManage showvminfo "WinServer" --machinereadable | findstr /B "nic"
nic1="intnet"
nic2="nat"
```

Inside the guest, you should see two adapters with different address ranges:

```
ipconfig /all
Ethernet      ...  10.0.0.50    <- lab network, talks to the manager
Ethernet 2    ...  10.0.2.15    <- NAT, internet
```

> **This is the mistake to avoid**
>
> If a machine has two adapters, **the first one is usually the one Windows and the agent default to**. Keep `10.0.0.x` as the primary interface. If the agent somehow binds to `10.0.2.x` it will try to reach the manager through NAT, fail, and you will spend an hour debugging a firewall that is not the problem.

3 The install script

You could copy a dozen commands by hand from the Wazuh docs. A script is better here for one reason: it tells you what it is doing at each step, and it fails loudly instead of silently.

The script is `Install-WazuhAgent.ps1`, sitting next to this document. It does the whole job, prints what it is doing, and finishes with a verdict you can trust.

3.1 What it does, in order

#	Step	Why it is there
1	Check it is elevated	Installing a service needs admin. Failing here with a clear message beats a confusing error three steps later.
2	Show agent identity	So you can see the name, manager and group before anything is written to disk. Mistyped agent names are annoying to fix later.
3	Test 1514 and 55000	Proves the path to the manager works *before* installing. An agent that cannot reach the manager installs fine and then reports nothing, which is a miserable thing to debug.
4	Download the MSI	Uses the NAT adapter added above. Fails with an explanation if there is no internet route.
5	Install silently	Writes a verbose MSI log so you can see exactly what happened if anything goes wrong.
6	Start the service	Named `WazuhSvc`, and set to start automatically so it survives a reboot.
7	Firewall rule	Inbound 1514 only, and only if it is not already there.
8	Read the log	Confirms whether registration actually succeeded, and quotes the error if not.
9	Show the config	Proves the manager IP and agent name landed in `ossec.conf` as you intended.

3.2 Running it on WinServer

Open an Administrator PowerShell on WinServer. If the script is on the host, put it on a VirtualBox shared folder, or copy it in. Then:

```
cd C:\Path\To\Script
.\Install-WazuhAgent.ps1 -AgentName winsrv-01 -AgentGroup windows,servers
```

If PowerShell blocks the script because of execution policy:

```
powershell -ExecutionPolicy Bypass -File .\Install-WazuhAgent.ps1 `
  -AgentName winsrv-01 -AgentGroup windows,servers
```

> **Why ExecutionPolicy Bypass is acceptable here, and only here**
>
> Execution policy is a safety net against running scripts you did not inspect. You are inspecting it, it is sitting on your own disk, and `Bypass` applies only to that one process. Do not set the machine policy to `Unrestricted`, and do not paste this into a shared profile.

3.3 Running it on WindowsClient

Same script, different name and group:

```
powershell -ExecutionPolicy Bypass -File .\Install-WazuhAgent.ps1 `
  -AgentName wincli-01 -AgentGroup windows,endpoints
```

Parameter	Use	Notes
`-AgentName`	Must be unique	Defaults to the hostname lowercased. Two agents with the same name is a real and confusing failure.
`-AgentGroup`	windows, servers / endpoints	Groups are **enforced** by the manager. They decide what configuration your agent receives.
`-ManagerIP`	Defaults to 10.0.0.40	Use the internal-network address, never the NAT one.
`-SkipDownload`	Reuse a local MSI	For when you have no internet route and sideloaded the file.
`-RegistrationPassword`	Only if you enabled key auth	Leave it out for the simple lab setup.

> **Groups versus tags — the classic beginner bug**
>
> **Groups** are enforced by the manager: an agent's configuration is assembled from the groups it belongs to. Use them for "everything in this group gets this hardening". **Tags** are free-form metadata for filtering and have no effect on configuration. Putting descriptive words in the group field, or expecting a tag to change behaviour, is the most common configuration mistake in a first SIEM lab.

3.4 What success looks like

```
=== Checking administrator rights ===
  [OK]   Running elevated

=== Testing the path to the manager ===
  [OK]   Port 1514 reachable at 10.0.0.40
  [OK]   Port 55000 reachable at 10.0.0.40

=== Installing silently ===
  [OK]   MSI returned 0 (success)

=== Setting the service to start automatically ===
  [OK]   WazuhSvc is Running, start type Automatic

  RESULT: agent is running and the manager is reachable.
```

4 Confirming it on the manager

A green result in the terminal is necessary but not sufficient. The proof is on the manager.

4.1 The dashboard

Open the Wazuh dashboard from the manager VM. The host cannot reach `10.0.0.40` directly, so either use the SIEM's own IP through your browser if you have a port forward, or browse from inside the SIEM VM. Then go to **Agents**.

You see	It means
`Active, green`	Working. This is the goal.
`Never connected`	Installed but could not reach the manager. Read the agent's own log.
`Pending / Disconnected`	Was connected, now is not. Common after the VM sleeps or the manager restarts.
`Not listed at all`	Did not enrol. Almost always port 55000 unreachable.

4.2 From the manager shell

On SIEM-Ubuntu, over SSH from Kali or a console:

```
ssh <user>@10.0.0.40
sudo /var/ossec/bin/agent_control -l
```

```
ID: 1, Name: kali-linux-2026.2-virtualbox-amd64, Active
ID: 2, Name: winsrv-01, Active
ID: 3, Name: wincli-01, Active
```

Two more commands worth knowing, both on the manager:

```
# force an agent to re-read its configuration
sudo /var/ossec/bin/agent_control -R 2
sudo /var/ossec/bin/agent_control -i 2

# see the last messages the agent sent
sudo /var/ossec/bin/agent_control -l 2
```

4.3 Prove data is actually flowing

An agent can show Active while sending nothing useful. Generate a real event and go looking for it. On the target, create a file in a monitored directory, or simply try and fail a logon a few times:

```
# on the target, generate a failed logon
runas /user:definitely-not-a-real-user cmd
```

Then in the dashboard, filter for it:

```
agent.name:winsrv-01 AND rule.level:>=5
```

Or use the raw Windows event ID, which is the tool you will reach for most once Windows is enrolled:

```
data.win.system.eventID:4625
```

> **Event ID 4625 is worth memorising now**
>
> Failed logon. Its siblings, 4624 successful logon, 4648 logon with explicit credentials, 4672 special privileges assigned, and 4720 account created, are the backbone of Windows account-attack detection. Learn these five and you can read a brute-force or lateral-movement attempt straight off the screen.

5 What went wrong while building this, and what it taught me

These are real bugs hit while preparing this lab on a real machine. Every one of them is a mistake that is easy to make and hard to diagnose from the error message alone.

5.1 The service is called WazuhSvc, not Wazuh

Every piece of documentation, blog post and ChatGPT answer will tell you to run `Get-Service Wazuh`. On Windows, the service is registered as `WazuhSvc`, with display name *Wazuh*. The error you get is unhelpful:

```
Set-Service : Service 'Wazuh (WazuhSvc)' cannot be configured due to the
following error: The specified service does not exist as an installed service.
```

Note what that message actually says: Wazuh *(WazuhSvc)*. It is telling you the answer in the error. Check the real name before you trust a guide:

```
Get-Service | Where-Object { $_.Name -match 'Wazuh|ossec' }
Name      DisplayName  Status  StartType
WazuhSvc  Wazuh        Running Automatic
```

5.2 The log is ossec.log, not wazuh-agent.log

On Linux the agent log is `/var/ossec/logs/ossec.log` and many guides reference `wazuh-agent.log`. On Windows the file that actually exists is:

```
C:\Program Files (x86)\ossec-agent\ossec.log
```

Looking for the wrong filename makes a working agent look broken. This is worth internalising as a habit: when something does not behave, confirm the artifact exists before concluding the software is at fault.

5.3 $args is a reserved PowerShell variable

Building the `msiexec` argument list, the natural name is `$args`. That is an automatic variable in PowerShell and assigning to it silently breaks the call. The install produced no error and no log, and no service. Rename it:

```
# wrong - silently does nothing useful
$args = @('/i', $msi, '/quiet', 'WAZUH_MANAGER=10.0.0.40')
Start-Process msiexec.exe -ArgumentList $args -Wait

# right
$msiArgs = @('/i', $msi, '/quiet', 'WAZUH_MANAGER=10.0.0.40')
Start-Process msiexec.exe -ArgumentList $msiArgs -Wait -PassThru
```

5.4 Check the real property names in the MSI

Rather than trusting documentation for the MSI property names, read them out of the installer itself. Decoding the file as Latin-1 and searching reveals exactly what it accepts:

```
python -c "
import re,pathlib
raw=pathlib.Path('wazuh-agent.msi').read_bytes().decode('latin1','ignore')
for h in sorted(set(re.findall(r'WAZUH[A-Z_]{2,30}',raw))): print(h)"
```

```
WAZUH_MANAGER
WAZUH_MANAGER_PORT
WAZUH_REGISTRATION_SERVER
WAZUH_REGISTRATION_PORT
WAZUH_AGENT_NAME
WAZUH_AGENT_GROUP
WAZUH_REGISTRATION_PASSWORD
WAZUH_REGISTRATION_CERTIFICATE
WAZUH_REGISTRATION_CA
WAZUH_REGISTRATION_KEY
```

That is the authoritative list. It also confirms that the modern, explicit form is `WAZUH_REGISTRATION_SERVER` and `WAZUH_REGISTRATION_PORT`, which is clearer than the older single `WAZUH_MANAGER` property.

5.5 Error 1208 means the enrolment port, not a broken agent

```
2026/09/30 23:12:38 wazuh-agent: ERROR: (1208):
Unable to connect to enrollment service at '[10.0.0.40]:55000'
```

This is the error you will see if the manager is down, if 55000 is firewalled, or if the agent is pointed at the wrong address. It is not a corrupt installation. Check in this order:

```
# 1. is the manager up?
sudo systemctl status wazuh-manager

# 2. from the target, can you reach both ports?
Test-NetConnection -ComputerName 10.0.0.40 -Port 55000
Test-NetConnection -ComputerName 10.0.0.40 -Port 1514

# 3. is the agent pointing at the right address?
Get-Content "C:\Program Files (x86)\ossec-agent\ossec.conf" |
  Select-String "<address>|<port>|agent_name"
```

5.6 A note on testing installs on the wrong machine

While preparing this I test-installed the agent on my own Windows host to verify the script end to end, then removed it completely: uninstall via the product code, delete the leftover `ossec-agent` directory and the firewall rule, and confirm the service is gone.

Verify on a throwaway, not on the machine you work on

Test end-to-end runs of installer scripts on a disposable VM. An agent on your daily machine ships its logs somewhere, adds a service, and opens a firewall rule, and cleaning that up properly takes more care than installing it did. If you must test on a real machine, record the product code before you start so you can uninstall precisely rather than guessing.

6 Configuring the SIEM, which is the real skill

Installing an agent is the easy part and honestly the least valuable. Configuring the manager to detect what matters is where the learning is.

6.1 Agent groups, in practice

Groups are how one manager serves different machines. Create them in the dashboard under **Agents → Groups**.

Group	Members	Configuration it should carry
`windows`	all Windows agents	Baseline event channels: logon events, service changes, account management, Sysmon if present.
`servers`	winsrv-01	Stricter: monitor service and scheduled-task changes, SMB configuration, local administrator list.
`endpoints`	wincli-01	Endpoint-flavoured: USB device insertion, browser extensions, large outbound transfers.

An agent in `windows,endpoints` receives the union of both groups' configuration. This is the first genuinely useful thing to get your hands on, because it is how a real shop manages a mixed estate without hand-editing every agent.

6.2 Turning on richer Windows telemetry

Stock Windows event channels are thin on process detail. Two cheap upgrades change what you can detect:

Sysmon

Microsoft's Sysinternals tool writes one consolidated log of process creation, network connections, and registry changes. Wazuh decodes it natively, so no custom parsing is needed.

```
# download from the official Sysinternals site, then:
Expand-Archive .\Sysmon.zip -DestinationPath C:\Sysmon
cd C:\Sysmon
.\Sysmon64.exe -i          # install with default config
.\Sysmon64.exe -c          # check the active config
```

It is genuinely easy to flood with volume, so a minimal include list is usually better than the default:

```
<Sysmon schemaversion="4.90">
  <EventFiltering>
    <ProcessCreate onmatch="include">
      <Image condition="is not any C:\Windows\System32\svchost.exe"/>
    </ProcessCreate>
    <NetworkConnect onmatch="include">
      <DestinationPort condition="is any 3389,445,5985,5986"/>
    </NetworkConnect>
    </EventFiltering>
</Sysmon>
```

Those destination ports are the interesting ones for your lab: RDP, SMB, and WinRM are what lateral movement actually uses.

> **Sysmon events arrive in Wazuh as win_eventchannel**
>
> They are not a separate data source, so you filter them the same way: `data.win.system.eventID:1` is process creation under Sysmon, whereas the equivalent stock channel is 4688. Knowing which of the two is feeding a rule is a question worth asking before you trust a detection.

6.3 The three queries to learn first

The dashboard search bar accepts Lucene syntax. These three cover most of what a beginner needs:

```
rule.level:>=10
# high and critical only. your daily triage starting point.

rule.level:>=8 AND agent.name:"winsrv-01"
# high alerts on one machine. AND must be uppercase.

data.win.system.eventID:4625
# failed logons. the brute-force detector.
```

6.4 Turning detections into a red/blue loop

Now the interesting part. From Kali, generate activity, then go to the dashboard and find it. This is the exercise that teaches you the most.

From Kali, on 10.0.0.30	What you should see	The lesson
`nmap -sV -p- --open 10.0.0.50`	Port scan or service-detection alerts, moderate level	Scans are noisy but visible. If yours were silent, the firewall profile is hiding them.
`crackmapexec smb 10.0.0.50 -u guest -p 'x'`	Failed logons, 4625, authentication failures	Repeated failures are what turns noise into an alert. One is an event, fifty is an attack.
`smbclient -L //10.0.0.50 -N`	Share enumeration, possibly a rule about anonymous access	Null-session share listing should be blocked. If it works, that is a real finding.
`Responder or a rogue SMB server`	NTLM relay or hash-capture alerts, higher severity	Protocol downgrade and relay are among the most common real-world attacks.
`winexe or PsExec to 10.0.0.50`	Service creation event 7045, then process creation	Remote service installation is a loud, reliable signal. This is why it is a favourite technique.

> **Do the loop in both directions**
>
> First, with the agent installed, confirm you can *see* the attack. Then harden the target, run it again, and confirm the gap is *closed* that you can no longer do it, or can do it but it is now detected. The second half is the part that teaches security. Finding a gap is half a job; proving the fix works is the other half, and it is the half people skip.

7 Reference

7.1 Quick command sheet

Task	Command
Is the agent running?	`Get-Service WazuhSvc`
Restart it	`Restart-Service WazuhSvc`
What is it configured with?	`Get-Content "C:\Program Files (x86)\ossec-agent\ossec.conf" | Select-String "<address>|<port>|agent_name"`
What is it doing?	`Get-Content "C:\Program Files (x86)\ossec-agent\ossec.log" -Tail 30`
Can it reach the manager?	`Test-NetConnection 10.0.0.40 -Port 1514`
Windows security events	`Get-WinEvent -FilterHashtable @{LogName='Security'; Id=4625} -MaxEvents 5`
Sysmon events	`Get-WinEvent -LogName "Microsoft-Windows-Sysmon/Operational" -MaxEvents 5`
Running processes with paths	`Get-CimInstance Win32_Process | Select-Object ProcessId, Name, CommandLine`
Listening ports and owners	`Get-NetTCPConnection -State Listen | Select-Object LocalPort, OwningProcess`
Local administrators	`Get-LocalGroupMember Administrators`
Uninstall completely	`msiexec /x {840F372A-2C20-49B9-A164-D8A14E8E8D47} /quiet`

7.2 Troubleshooting matrix

Symptom	Most likely cause	Fix
`Service 'Wazuh' does not exist`	Wrong service name	It is `WazuhSvc`. Check with `Get-Service | ? {$_.Name -match 'Wazuh'}`
`MSI exits 0 but no service`	Broken argument list	Do not name the variable `$args`. Check the verbose log at `%TEMP%\wazuh-msi.log`
`Error 1208 enrolment`	55000 unreachable	Manager down, or blocked. Test the port from the target.
`Install hangs on download`	No internet route	Add the NAT adapter, or sideload the MSI and use `-SkipDownload`
`Never connected in dashboard`	Wrong manager IP	Must be the internal-network address, never the NAT one.

`Duplicate agent name`	Two machines, same name	Names must be unique. Rename in the dashboard and reinstall.
`Active but no alerts`	Not enough data yet	Wait for the first FIM scan, or generate a test event and filter for it.
`Config changes not applied`	Agent has not re-read	`agent_control -R <id>` then `-i <id>` on the manager.
`Too many low-level alerts`	Sysmon too broad	Trim the include list in the Sysmon config.

7.3 Vocabulary

Term	Meaning
Manager	The server that receives, stores and analyses everything. Your SIEM-Ubuntu.
Agent	The software on a monitored endpoint. This document installs these.
Indexer	The storage engine behind the manager, on port 9200. Do not expose it.
Dashboard	The web interface on port 443 where you read alerts.
authd	The enrolment service on port 55000 that hands agents their key.
Enrolment	First-time registration of an agent with the manager.
Group	A manager-enforced set of agents sharing configuration.
Tag	Free-form metadata for filtering. No effect on configuration.
Rule	A detection pattern. Produces an alert when it matches.
Level	Severity 0–15. Sort by it; 10 and above means investigate now.
FIM	File Integrity Monitoring. Detects changes to important files.
SCA	Security Configuration Assessment. Checks against CIS-style baselines.
MITRE ATT&CK	The tactic and technique framework Wazuh tags alerts with.

7.4 The complete script, for reference

This is the whole of `Install-WazuhAgent.ps1` as it was tested. You do not need to read it to use it, but reading it is the fastest way to understand exactly what the install does, and it is worth copying into your own notes.

```powershell
<#
    Install-WazuhAgent.ps1
    One-file Wazuh agent installer for the red/blue lab.

    Run this on the TARGET machine (WinServer 2022 or WindowsClient Win11)
    from an Administrator PowerShell.

    Examples:
      .\Install-WazuhAgent.ps1 -AgentName winsrv-01 -AgentGroup windows,servers
      .\Install-WazuhAgent.ps1 -AgentName wincli-01 -AgentGroup windows,endpoints

    It does the whole job and then tells you whether it worked.
#>

[CmdletBinding()]
param(
    # Wazuh manager (your SIEM-Ubuntu VM) on the isolated lab network.
    [string]$ManagerIP = '10.0.0.40',

    # Agent name. Must be unique. Defaults to the machine hostname.
    [string]$AgentName,

    # Groups. These are ENFORCED on the manager - they decide what config
    # your agent gets. Do not use them for descriptive tags.
    [string]$AgentGroup = 'windows',

    [string]$MsiUrl = 'https://packages.wazuh.com/4.x/windows/wazuh-agent-4.14.8-1.msi',

    # Registration port (authd) on the manager. The agent also needs 1514 for events.
    [int]$ManagerPort = 1514,
    [int]$RegistrationPort = 55000,

    # Optional: pre-shared key, if you have enabled API key auth on the manager.
    # Leave empty for the simple, recommended lab setup.
    [string]$RegistrationPassword,

    # Set -SkipDownload if you already copied the MSI onto the machine.
    [switch]$SkipDownload
)

$ErrorActionPreference = 'Stop'

function Write-Step  { param($m) Write-Host "`n=== $m ===" -ForegroundColor Cyan }
function Write-Ok    { param($m) Write-Host "  [OK]   $m" -ForegroundColor Green }
function Write-Bad   { param($m) Write-Host "  [FAIL] $m" -ForegroundColor Red }
function Write-Info  { param($m) Write-Host "  [info] $m" -ForegroundColor Gray }

# ---------------------------------------------------------------- 0. admin
Write-Step 'Checking administrator rights'
$isAdmin = ([Security.Principal.WindowsPrincipal] `
    [Security.Principal.WindowsIdentity]::GetCurrent()
    ).IsInRole([Security.Principal.WindowsBuiltInRole]::Administrator)

if (-not $isAdmin) {
    Write-Bad 'This script must run as Administrator.'
    Write-Info 'Right-click PowerShell -> Run as administrator, then run it again.'
    exit 1
}
Write-Ok 'Running elevated'
```

```powershell
# ------------------------------------------------- 1. name and local identity
if (-not $AgentName) { $AgentName = $env:COMPUTERNAME.ToLower() }
Write-Step 'Agent identity'
Write-Info "Agent name : $AgentName"
Write-Info "Manager     : $ManagerIP"
Write-Info "Group(s)    : $AgentGroup"

$ip = Get-NetIPAddress -AddressFamily IPv4 |
      Where-Object { $_.IPAddress -notlike '127.*' -and
                     $_.IPAddress -notlike '169.254.*' } |
      Select-Object -First 1
if ($ip) { Write-Info "This host   : $($ip.IPAddress) on $($ip.InterfaceAlias)" }

# ------------------------------------------------- 2. can we reach the manager?
Write-Step 'Testing the path to the manager'
foreach ($port in @($ManagerPort, $RegistrationPort)) {
    $t = Test-NetConnection -ComputerName $ManagerIP -Port $port -WarningAction SilentlyContinue
    if ($t.TcpTestSucceeded) { Write-Ok "Port $port reachable at $ManagerIP" }
    else {
        Write-Bad "Cannot reach $ManagerIP`:$port"
        Write-Info "The agent needs $ManagerPort (events) and $RegistrationPort (enrolment)."
        Write-Info 'On the manager, both are opened by the Wazuh service itself.'
    }
}

# ------------------------------------------------------- 3. download the MSI
$msi = Join-Path $env:TEMP 'wazuh-agent.msi'

if ($SkipDownload) {
    Write-Step 'Skipping download'
} else {
    Write-Step 'Downloading the Wazuh agent'
    Write-Info $MsiUrl
    try {
        [Net.ServicePointManager]::SecurityProtocol = [Net.SecurityProtocolType]::Tls12
        Invoke-WebRequest -Uri $MsiUrl -OutFile $msi -UseBasicParsing
        $kb = [math]::Round((Get-Item $msi).Length / 1KB)
        Write-Ok "Downloaded ($kb KB) -> $msi"
    } catch {
        Write-Bad "Download failed: $($_.Exception.Message)"
        Write-Info 'No internet on this NIC? Copy the MSI across manually and re-run with -
SkipDownload'
        exit 1
    }
}

# ------------------------------------------------------------ 4. install it
Write-Step 'Installing silently'
Write-Info 'This takes 30-60 seconds.'

$log = Join-Path $env:TEMP 'wazuh-msi.log'
# NOTE: do not name this $args - that is a PowerShell automatic variable.
$msiArgs = @(
    '/i', $msi, '/quiet', '/norestart', '/l*v', $log,
    "WAZUH_MANAGER=$ManagerIP",
    "WAZUH_MANAGER_PORT=$ManagerPort",
    "WAZUH_REGISTRATION_SERVER=$ManagerIP",
    "WAZUH_REGISTRATION_PORT=$RegistrationPort",
    "WAZUH_AGENT_NAME=$AgentName",
    "WAZUH_AGENT_GROUP=$AgentGroup"
)
```

```
if ($RegistrationPassword) {
    $msiArgs += "WAZUH_REGISTRATION_PASSWORD=$RegistrationPassword"
}
$p = Start-Process msiexec.exe -ArgumentList $msiArgs -Wait -PassThru

switch ($p.ExitCode) {
    0       { Write-Ok 'MSI returned 0 (success)' }
    3010    { Write-Ok 'Installed, reboot required' }
    1618    { Write-Bad 'Another install is in progress. Reboot and retry.'; exit 1 }
    1638    { Write-Bad 'A newer version is already installed.'; exit 1 }
    default { Write-Bad "MSI failed, exit code $($p.ExitCode). Log: $log"; exit 1 }
}

# ------------------------------------------- 5. service auto-start + running
# NOTE: the service is called WazuhSvc, NOT "Wazuh". Getting this name wrong
# is the single most common failure when scripting the Wazuh agent on Windows.
Write-Step 'Setting the service to start automatically'
$svc = Get-Service -Name 'WazuhSvc' -ErrorAction SilentlyContinue
if (-not $svc) {
    Write-Bad 'Service "WazuhSvc" not found after install. Check the MSI log.'
    exit 1
}
Set-Service -Name 'WazuhSvc' -StartupType Automatic
Start-Service -Name 'WazuhSvc'
Start-Sleep -Seconds 5
$svc = Get-Service -Name 'WazuhSvc'
if ($svc.Status -eq 'Running') { Write-Ok "WazuhSvc is $($svc.Status), start type
$($svc.StartType)" }
else { Write-Bad "WazuhSvc is $($svc.Status)" }

# ------------------------------------------------------------ 6. firewall rule
Write-Step 'Opening the firewall for the agent'
$existing = Get-NetFirewallRule -DisplayName 'Wazuh Agent 1514' -ErrorAction SilentlyContinue
if ($existing) {
    Write-Ok 'Rule already present'
} else {
    New-NetFirewallRule -DisplayName 'Wazuh Agent 1514' `
        -Direction Inbound -Action Allow -Protocol TCP `
        -LocalPort 1514 -Profile Any | Out-Null
    Write-Ok 'Created inbound rule on TCP 1514'
}

# ------------------------------------------------------ 7. verify registration
# The agent writes to ossec.log, NOT wazuh-agent.log. The latter only appears
# on Linux. Looking for the wrong file is why "I can't see the log" is common.
Write-Step 'Reading the agent log for registration'
$agentLog = 'C:\Program Files (x86)\ossec-agent\ossec.log'

Start-Sleep -Seconds 12
if (Test-Path $agentLog) {
    $tail = Get-Content $agentLog -Tail 40

    $registered = $tail | Select-String 'Enrolled new agent|Registered successfully|Valid
registration'
    if ($registered) {
        $registered | Select-Object -Last 3 | ForEach-Object { Write-Ok $_.Line.Trim() }
    } else {
        Write-Info 'No successful registration line yet.'
    }

    if ($tail | Select-String 'ERROR') {
```

```
        Write-Bad 'Errors in the log:'
        ($tail | Select-String 'ERROR') | Select-Object -Last 5 |
            ForEach-Object { Write-Info $_.Line.Trim() }
        Write-Info 'Error 1208 = cannot reach the enrolment service on 55000.'
        Write-Info 'That is almost always the manager being down, or a firewall in the way.'
    }
} else {
    Write-Info "Log not found yet at $agentLog"
}

# ------------------------------------------------------------ 8. configuration
# The real settings live in ossec.conf (wazuh-agent.conf is for
# osquery.conf-style local overrides and is usually not what you want to read).
Write-Step 'What the agent is actually configured with'
$conf = 'C:\Program Files (x86)\ossec-agent\ossec.conf'
if (Test-Path $conf) {
    Get-Content $conf | Select-String '<server>|<address>|<port>|<agent_name>|<labels>' |
        Select-Object -First 10 | ForEach-Object { Write-Info $_.Line.Trim() }
} else {
    Write-Info "Config not found at $conf"
}

# ------------------------------------------------------------------ summary
Write-Step 'Summary'
$final = Test-NetConnection -ComputerName $ManagerIP -Port $ManagerPort -WarningAction
SilentlyContinue
$svc = Get-Service -Name 'WazuhSvc'

Write-Host ""
Write-Host "  Agent name : $AgentName" -ForegroundColor White
Write-Host "  Service    : $($svc.Status) / $($svc.StartType)" -ForegroundColor White
Write-Host "  Manager    : $ManagerIP (port $ManagerPort reachable: $($final.TcpTestSucceeded))" -
ForegroundColor White
Write-Host "  Installed  : $((Get-Item $msi -ErrorAction SilentlyContinue).LastWriteTime)" -
ForegroundColor Gray
Write-Host ""

if ($svc.Status -eq 'Running' -and $final.TcpTestSucceeded) {
    Write-Host '  RESULT: agent is running and the manager is reachable.' -ForegroundColor Green
    Write-Host '  Next: open the Wazuh dashboard -> Agents. This machine should turn Active' -
ForegroundColor Green
    Write-Host "  within about 30 seconds. Look for '$AgentName'." -ForegroundColor Green
} else {
    Write-Host '  RESULT: something is not right yet. See the [FAIL] lines above.' -ForegroundColor
Red
    Write-Host '  Most common cause: wrong ManagerIP, or the manager is not running.' -
ForegroundColor Red
}

Write-Host ""
Write-Host '  Local log to watch: ' -NoNewline
Write-Host 'C:\Program Files (x86)\ossec-agent\ossec.log' -ForegroundColor Cyan
Write-Host '  (ossec.log, not wazuh-agent.log - that filename is Linux-only)' -ForegroundColor
DarkGray
Write-Host ''
```

7.5 Where to go next

- Your **Volume 2** guide covers WinServer hardening. Every control it configures will now generate evidence in the dashboard, which makes it far more useful to work through.
- Your **Volume 1** guide explains the dashboard, filtering and the rule taxonomy. Read it before you start triaging, so the columns mean something to you.
- Add **Sysmon** next. It is the single biggest detection improvement per minute of effort.
- Then build the attack loop from section 6.4, and harden against what you find.

> **A closing note on where to point this**
>
> Everything above runs against VirtualBox VMs on your own laptop, on a segment with no route to or from anything real. That is the right place to practise offensive technique, and the reason it is worth keeping isolated. Scanning, credential testing and exploitation against machines you do not own is a different activity governed by law that varies by country, and consent that is not implied by a machine being reachable. The skills transfer completely; the target has to be yours.